

California-Housing

June 4, 2026

1 California Housing Project (Chapter-2)

2 1. The Big Picture

2.1 1.1 Business Objective

The raw estimation is not enough! we should know where our output is going to be used?

concern	description
CHAIN	our <i>output</i> enters another system which determine whether investing in that area is profitable or not
NOTICE	if our system fails the next system would most definitely fail which results in huge failure
Schema	data -> our_model -> investing_model

2.2 1.2 Current Solution

Currently, experts manually estimate the price. They collect a lot of data and when they can't make an accurate estimate, they give a price with a 10 to 30 percent error.

2.3 1.3 Framing

Now you need to categorize the problem (the Perspective)

concern	reason
Supervised Learning	We Have actual prices
Regression , ...	We'll estimate a number as output
Batch	<You Say!>

2.4 1.4 Performance Metrics

In simple terms, here is why we prefer **RMSE** over **MAE** for this project:

- Heavier Penalty for Big Mistakes: RMSE squares the errors. This means if the model is off by a huge amount on one house, the “penalty” is much higher than in MAE. In real estate, a massive miscalculation can lead to a financial disaster, so we want to avoid big errors at all costs.

- Mathematical Smoothness: RMSE is easier for computers to calculate and optimize because its mathematical curve is smooth. MAE has a sharp “V” shape that makes it slightly harder for some algorithms to find the perfect solution.

Summary:

- RMSE = “I really hate big errors.”
- MAE = “I treat all errors proportionally.”

Since we want to avoid huge pricing disasters in California, **RMSE is our best friend here**

3 2. Get the Data

3.1 2.1 Download Data (with proper VPN)

```
[ ]: !wget "https://storage.googleapis.com/kaggle-data-sets/5227/7876/bundle/archive.
↪zip"
```

```
--2026-06-03 11:32:35-- https://storage.googleapis.com/kaggle-data-
sets/5227/7876/bundle/archive.zip
Resolving storage.googleapis.com (storage.googleapis.com)... 50.7.5.83
Connecting to storage.googleapis.com (storage.googleapis.com)|50.7.5.83|:443...
connected.
HTTP request sent, awaiting response... 403 Forbidden
2026-06-03 11:32:36 ERROR 403: Forbidden.
```

3.2 2.2 Load Data

```
[ ]: import pandas as pd
import os

def load_housing_data(housing_path="data"):
    csv_path = os.path.join(housing_path, "housing_data.csv")
    return pd.read_csv(csv_path)
```

```
[ ]: housing_df = load_housing_data()

# Smoke Test
housing_df.head()
```

```
[ ]:      longitude  latitude  housing_median_age  total_rooms  total_bedrooms  \
0      -122.23     37.88           41.0           880.0           129.0
1      -122.22     37.86           21.0          7099.0          1106.0
2      -122.24     37.85           52.0          1467.0           190.0
3      -122.25     37.85           52.0          1274.0           235.0
4      -122.25     37.85           52.0          1627.0           280.0

population  households  median_income  median_house_value  ocean_proximity
```

0	322.0	126.0	8.3252	452600.0	NEAR BAY
1	2401.0	1138.0	8.3014	358500.0	NEAR BAY
2	496.0	177.0	7.2574	352100.0	NEAR BAY
3	558.0	219.0	5.6431	341300.0	NEAR BAY
4	565.0	259.0	3.8462	342200.0	NEAR BAY

4 3. Explore the Data

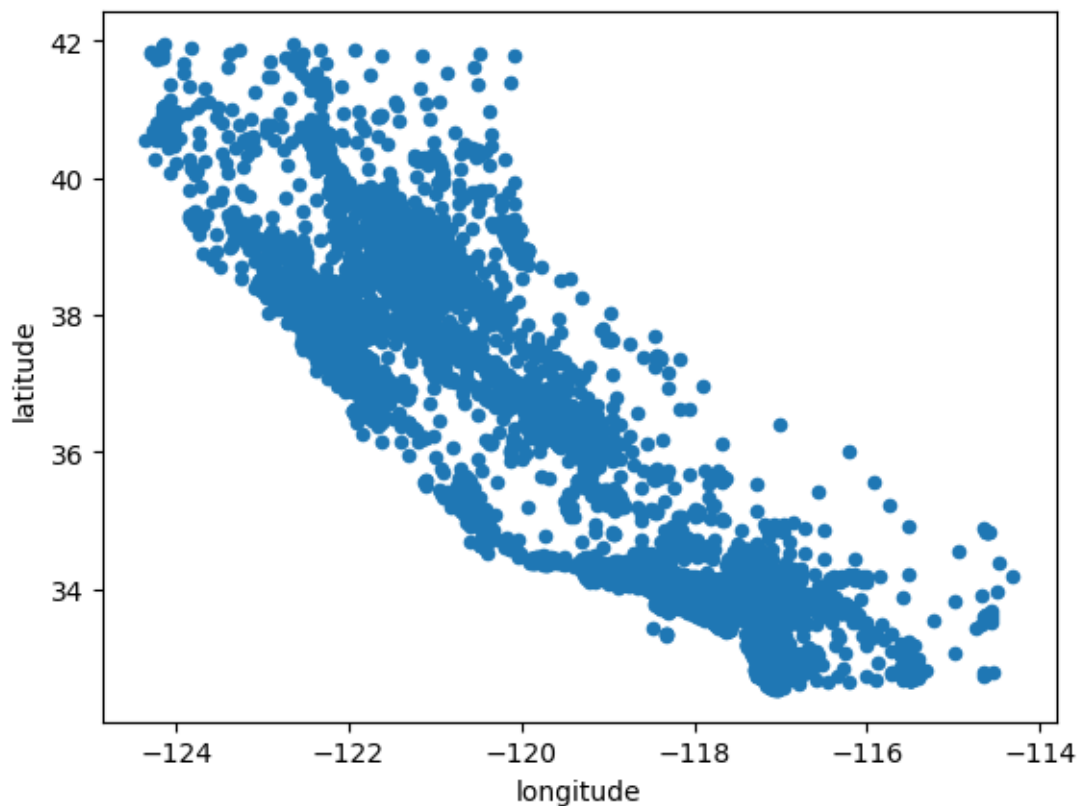
```
[ ]: # to make sure we just explore
housing = housing_df.copy()
```

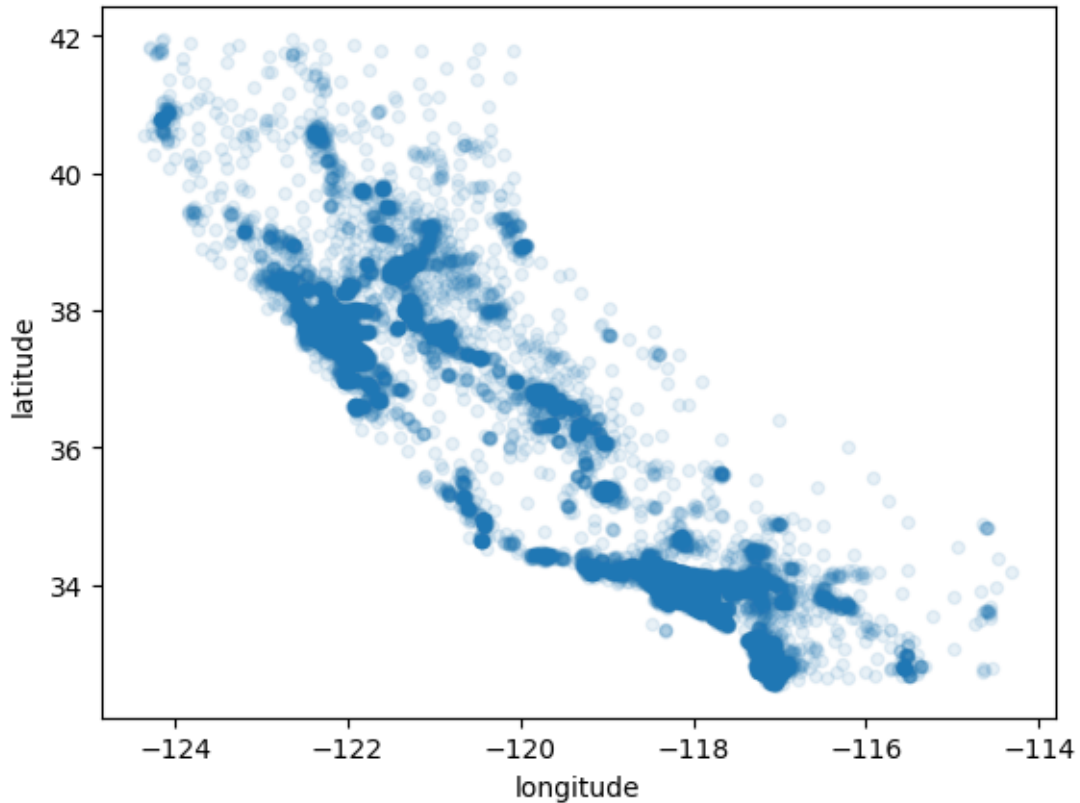
4.1 3.1 Visualizing Geographical Data

This code plots the latitude and longitude as X and Y on a graph. The output is just a bunch of black dots in the shape of a map of California, but **because the dots overlap**, we don't know where it's busy and where it's quiet.

```
[ ]: housing.plot(kind="scatter", x="longitude", y="latitude")
housing.plot(kind="scatter", x="longitude", y="latitude", alpha=0.1)
```

```
[ ]: <Axes: xlabel='longitude', ylabel='latitude'>
```





4.2 3.2 Looking for Correlations: Standard Coefficient Correlation

Now we need to understand the relationships between the variables in terms of numbers and formulas. The book uses the Pearson Correlation Coefficient to see how much each characteristic is related to the median house value.

NOTICE: number +1 determines positive correlation, -1 negative correlation, 0 determines lack of linear-correlation

```
[ ]: corr_matrix = housing_df.corr(numeric_only=True)
     corr_matrix["median_house_value"].sort_values(ascending=False)
```

```
[ ]: median_house_value    1.000000
     median_income         0.688075
     total_rooms           0.134153
     housing_median_age     0.105623
     households             0.065843
     total_bedrooms         0.049686
     population            -0.024650
     longitude              -0.045967
     latitude               -0.144160
     Name: median_house_value, dtype: float64
```

4.2.1 Experimenting with Attribute Combinations (leads to Feature Engineering)

```
[ ]: housing["rooms_per_household"] = housing["total_rooms"] / housing["households"]
housing["bedrooms_per_room"] = housing["total_bedrooms"] /
    ↪housing["total_rooms"]
housing["population_per_household"] = housing["population"] /
    ↪housing["households"]

# Re-run the correlation matrix
corr_matrix = housing.drop(columns=["ocean_proximity"]).corr()
print(corr_matrix["median_house_value"].sort_values(ascending=False))
```

```
median_house_value      1.000000
median_income           0.688075
rooms_per_household     0.151948
total_rooms             0.134153
housing_median_age      0.105623
households              0.065843
total_bedrooms          0.049686
population_per_household -0.023737
population              -0.024650
longitude               -0.045967
latitude                -0.144160
bedrooms_per_room       -0.255880
Name: median_house_value, dtype: float64
```

4.3 3.4 Scatter Matrix Diagram

To visualize these relationships, the book uses a pie chart tool to plot several important features against each other:

```
[ ]: from pandas.plotting import scatter_matrix
attributes = ["median_house_value", "median_income", "total_rooms",
    ↪"housing_median_age"]

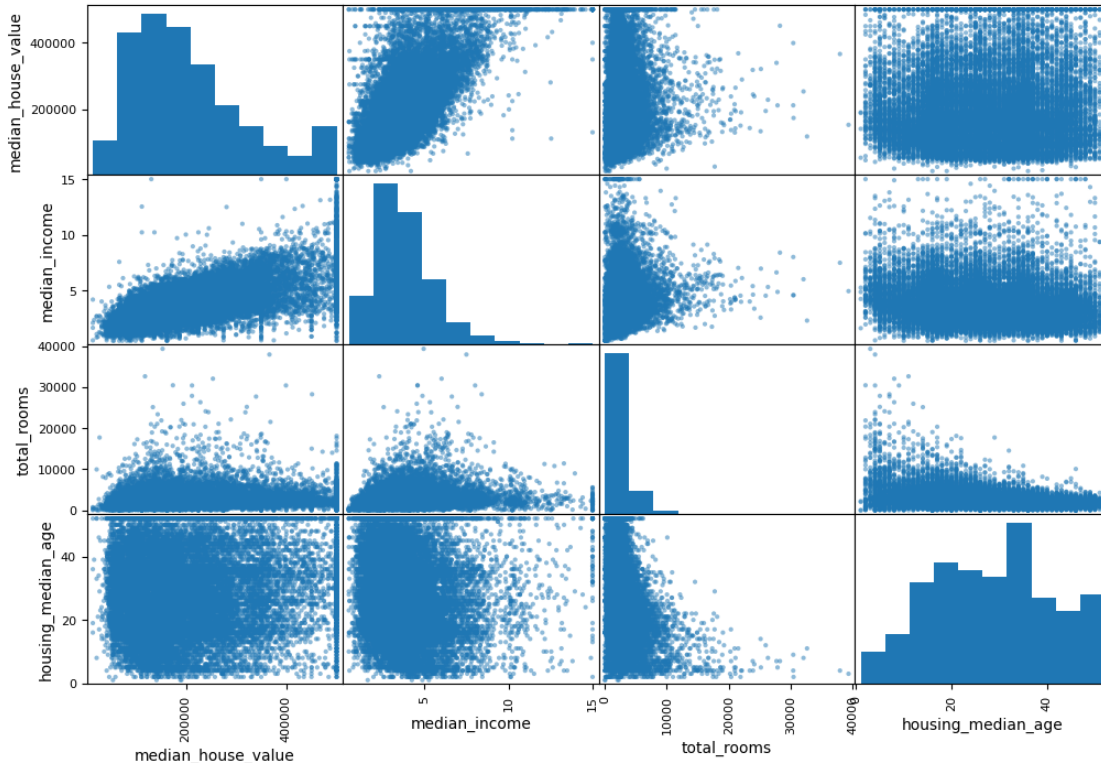
scatter_matrix(housing_df[attributes], figsize=(12, 8))

[ ]: array([[<Axes: xlabel='median_house_value', ylabel='median_house_value'>,
    <Axes: xlabel='median_income', ylabel='median_house_value'>,
    <Axes: xlabel='total_rooms', ylabel='median_house_value'>,
    <Axes: xlabel='housing_median_age', ylabel='median_house_value'>],
    [<Axes: xlabel='median_house_value', ylabel='median_income'>,
    <Axes: xlabel='median_income', ylabel='median_income'>,
    <Axes: xlabel='total_rooms', ylabel='median_income'>,
    <Axes: xlabel='housing_median_age', ylabel='median_income'>],
    [<Axes: xlabel='median_house_value', ylabel='total_rooms'>,
    <Axes: xlabel='median_income', ylabel='total_rooms'>,
    <Axes: xlabel='total_rooms', ylabel='total_rooms'>],
    [<Axes: xlabel='median_house_value', ylabel='housing_median_age'>,
    <Axes: xlabel='median_income', ylabel='housing_median_age'>,
    <Axes: xlabel='total_rooms', ylabel='housing_median_age'>,
    <Axes: xlabel='housing_median_age', ylabel='housing_median_age'>]])
```

```

<Axes: xlabel='housing_median_age', ylabel='total_rooms'>],
[<Axes: xlabel='median_house_value', ylabel='housing_median_age'>,
<Axes: xlabel='median_income', ylabel='housing_median_age'>,
<Axes: xlabel='total_rooms', ylabel='housing_median_age'>,
<Axes: xlabel='housing_median_age', ylabel='housing_median_age'>]],
dtype=object)

```



In Short here are the 3 main takeaways from that matrix:

- **median_income** is your goldmine: The tight upward trend in the income vs. house value plot confirms it is your strongest predictor by far.
- The data has artificial caps: The solid horizontal line at the very top (\$500,000) shows where the data was capped. You'll likely want to remove these districts so your model doesn't learn this fake rule.
- Raw totals are mostly useless: The messy, unstructured "clouds" for attributes like total_rooms show why raw counts don't correlate well with price. This is why you need the engineered ratios (like rooms_per_household) instead.

4.3.1 Feature Selection & Engineering Strategy

based on these strategies:

- Keep As-Is:

- `median_income`: Primary predictor (strongest linear signal).
- `longitude` & `latitude`: Crucial for non-linear geographical patterns.
- `housing_median_age`: Provides a stable, independent signal.
- Feature Engineering (Combine Low-Correlation Attributes):
 - Raw columns like `total_rooms`, `total_bedrooms`, and `population` have near-zero correlations because they are aggregate district totals.
 - Action: Transform them into **household-specific ratios** to unlock stronger signals:
 - * `rooms_per_household = total_rooms / households`
 - * `bedrooms_per_room = total_bedrooms / total_rooms`
 - * `population_per_household = population / households`
- Categorical Handling:
 - `ocean_proximity`: Keep it. It will be encoded into numerical values in the next stage since proximity to the coast is a major price driver.

5 4. Prepare the Data for Machine Learning Algorithms

1. Fix Missing Values (Imputation) The `total_bedrooms` column has missing values (NaN). We'll use `SimpleImputer` to replace them with the median of that column.

Note: Imputer only works on numerical columns, so we drop the text column `ocean_proximity` for this part.

2. Handle Text Categories (One-Hot Encoding) `ocean_proximity` is categorical text (e.g., <1H OCEAN, INLAND). ML models need numbers. We use `OneHotEncoder` to turn these categories into binary dummy variables (1s and 0s).
3. Feature Scaling (Standardization) ML algorithms choke when features have wildly different ranges (e.g., `total_rooms` goes up to tens of thousands, while `median_income` goes from 0.5 to 15). We use `StandardScaler` so every numerical feature has a mean of 0 and a variance of 1.

5.1 4.1 Separate Predictors and Target Labels

Before applying any transformations, we must separate our target variable (`median_house_value`) from our predictors. This ensures we don't accidentally modify or scale our target values, which must remain in their original form for loss evaluation.

We assume `housing_df` (created via stratified sampling in Step 2) is already loaded in the environment.

```
[ ]: # Separate features and target labels
housing = housing_df.drop("median_house_value", axis=1)
housing_labels = housing_df["median_house_value"].copy()
```

```
print(f"Predictors shape: {housing.shape}")
print(f"Labels shape: {housing_labels.shape}")
```

```
Predictors shape: (20640, 9)
Labels shape: (20640,)
```

5.2 4.2 Custom Transformer for Feature Engineering

To make our preprocessing pipeline fully reproducible and capable of handling future raw data dataframes, we wrap our engineered attributes into a custom Scikit-Learn transformer.

By inheriting from `BaseEstimator` and `TransformerMixin`, our class automatically acquires the `fit_transform()` method. We hardcode column index integers (3, 4, 5, 6) because pipelines often convert DataFrames into raw NumPy arrays mid-execution.

```
[ ]: import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin

# Column indices based on the original housing DataFrame structure:
# 3: total_rooms, 4: total_bedrooms, 5: population, 6: households
rooms_ix, bedrooms_ix, population_ix, households_ix = 3, 4, 5, 6

class CombinedAttributesAdder(BaseEstimator, TransformerMixin):
    def __init__(self, add_bedrooms_per_room=True):
        self.add_bedrooms_per_room = add_bedrooms_per_room

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        # Calculate engineered ratios from raw counts
        rooms_per_household = X[:, rooms_ix] / X[:, households_ix]
        population_per_household = X[:, population_ix] / X[:, households_ix]

        if self.add_bedrooms_per_room:
            bedrooms_per_room = X[:, bedrooms_ix] / X[:, rooms_ix]
            # Horizontally stack (glue) new features onto the end of the input
            ↪array
            return np.c_[X, rooms_per_household, population_per_household, ↪
            ↪bedrooms_per_room]
        else:
            return np.c_[X, rooms_per_household, population_per_household]
```

5.3 4.3 Constructing Preprocessing Pipelines via ColumnTransformer

We now assemble our complete preprocessing pipeline. Because datasets contain a mixture of data types, we split our workflow into two parallel branches that merge back together before outputting:

1. Numerical Pipeline (num_pipeline): * `SimpleImputer(strategy="median")`: Fills any missing

(NaN) values in numerical columns (like total_bedrooms) using the training set median.

- `CombinedAttributesAdder()`: Dynamically executes our custom feature engineering class defined above.
 - `StandardScaler()`: Standardizes features by shifting means to 0 and scaling variance to 1.
2. Categorical Pipeline: - `OneHotEncoder()`: Converts text categories (like ocean_proximity) into individual sparse matrix binary columns (0s and 1s).

```
[ ]: from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

# Isolate numerical columns to determine feature names for the pipeline split
housing_num = housing.drop(columns=["ocean_proximity"])
num_attribs = list(housing_num)
cat_attribs = ["ocean_proximity"]

# Sub-pipeline for handling all numerical features
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy="median")),
    ('attribs_adder', CombinedAttributesAdder(add_bedrooms_per_room=True)),
    ('std_scaler', StandardScaler()),
])

# Master ColumnTransformer to apply specific steps to specific columns
full_pipeline = ColumnTransformer([
    ("num", num_pipeline, num_attribs),
    ("cat", OneHotEncoder(), cat_attribs),
])
```

5.4 4.7 Running the Full Preprocessing Pipeline

With our pipeline architecture defined, we call `.fit_transform()` on our raw training predictors. This single line fits parameters (calculating column medians, means, and variances) only on the training data and returns the completed, processed dataset as a clean array ready for machine learning algorithms.

```
[ ]: housing_prepared = full_pipeline.fit_transform(housing)

# Verify the changes in dataset shape
print(f"Original shape: {housing.shape}")
print(f"Prepared shape: {housing_prepared.shape}")
print("\nFirst row sample of prepared data:")
print(housing_prepared[0])
```

Original shape: (20640, 9)

Prepared shape: (20640, 16)

First row sample of prepared data:

```
[-1.32783522  1.05254828  0.98214266 -0.8048191  -0.97247648 -0.9744286
 -0.97703285  2.34476576  0.62855945 -0.04959654 -1.02998783  0.
  0.          0.          1.          0.          ]
```

6 5. Select and Train a Model

6.1 5.1 Train and Evaluate a Linear Regression Model

We start with the simplest baseline model: a standard Linear Regression model. We will fit the model using our prepared features and target labels, and then evaluate its performance using Root Mean Squared Error (RMSE).

```
[ ]: from sklearn.linear_model import LinearRegression
     from sklearn.metrics import mean_squared_error

     # Initialize and fit the model
     lin_reg = LinearRegression()
     lin_reg.fit(housing_prepared, housing_labels)

     # Let's try it out on a few instances from the training set
     some_data = housing.iloc[:5]
     some_labels = housing_labels.iloc[:5]
     some_data_prepared = full_pipeline.transform(some_data)

     print("Predictions:", lin_reg.predict(some_data_prepared))
     print("Labels:", list(some_labels))

     # Measure the RMSE on the whole training set
     housing_predictions = lin_reg.predict(housing_prepared)
     lin_mse = mean_squared_error(housing_labels, housing_predictions)
     lin_rmse = np.sqrt(lin_mse)
     print(f"\nLinear Regression Training RMSE: ${lin_rmse:,.2f}")
```

```
Predictions: [406122.44317288 431667.18043819 377259.09665099 319150.08569981
 252750.08280698]
```

```
Labels: [452600.0, 358500.0, 352100.0, 341300.0, 342200.0]
```

```
Linear Regression Training RMSE: $68,286.13
```

Analyze Linear Regression Results (Underfitting) An RMSE of around \$68,000 means the model's predictions are, on average, off by \$68,000. Considering most districts' median house values range between \$120,000 and \$265,000, a \$68,000 error is quite high.

This is a classic example of a model underfitting the training data. The features do not provide enough information, or the linear model is simply not powerful enough to capture the underlying patterns.

Let's try a more complex, non-linear model: a Decision Tree Regressor.

6.2 5.2 Train and Evaluate a Decision Tree Regressor

```
[ ]: from sklearn.tree import DecisionTreeRegressor

# Initialize and fit the Decision Tree
tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(housing_prepared, housing_labels)

# Measure the RMSE on the training set
tree_predictions = tree_reg.predict(housing_prepared)
tree_mse = mean_squared_error(housing_labels, tree_predictions)
tree_rmse = np.sqrt(tree_mse)
print(f"Decision Tree Training RMSE: ${tree_rmse:,.2f}")
```

Decision Tree Training RMSE: \$0.00

6.2.1 Better Evaluation Using K-Fold Cross-Validation

Wait, an RMSE of \$0.00? Did we just build a flawless model?

Highly unlikely. It's much more probable that the model badly overfit the data. To confirm this without touching our precious test set yet, we need to perform K-Fold Cross-Validation. We will split the training set into 10 distinct subsets (folds), train the model 10 times (picking a different fold for evaluation each time), and average the results.

```
[ ]: from sklearn.model_selection import cross_val_score

def display_scores(scores):
    print("Scores:", [f"${score:,.2f}" for score in scores])
    print(f"Mean Score: ${scores.mean():,.2f}")
    print(f"Standard Deviation: ${scores.std():,.2f}")

# Cross-validation for Decision Tree
# Note: utility functions expect a utility function (greater is better) rather
#       than a cost function,
# so the scoring function is actually the opposite of the MSE.
tree_scores = cross_val_score(tree_reg, housing_prepared, housing_labels,
                              scoring="neg_mean_squared_error", cv=10)
tree_rmse_scores = np.sqrt(-tree_scores)

print("--- Decision Tree Cross-Validation ---")
display_scores(tree_rmse_scores)

# Let's cross-validate the Linear Regression model too for comparison
lin_scores = cross_val_score(lin_reg, housing_prepared, housing_labels,
                              scoring="neg_mean_squared_error", cv=10)
lin_rmse_scores = np.sqrt(-lin_scores)

print("\n--- Linear Regression Cross-Validation ---")
```

```
display_scores(lin_rmse_scores)
```

--- Decision Tree Cross-Validation ---

Scores: ['\$117,611.16', '\$72,276.94', '\$83,022.59', '\$74,950.41', '\$89,977.64', '\$77,719.87', '\$68,918.46', '\$99,331.91', '\$94,816.63', '\$75,710.10']

Mean Score: \$85,433.57

Standard Deviation: \$14,327.78

--- Linear Regression Cross-Validation ---

Scores: ['\$84,183.66', '\$61,191.53', '\$86,743.61', '\$62,286.73', '\$80,537.26', '\$68,918.59', '\$52,504.86', '\$90,904.23', '\$77,675.09', '\$53,940.95']

Mean Score: \$71,888.65

Standard Deviation: \$13,247.67

6.3 5.3 Evaluate a Random Forest Regressor

The Cross-Validation numbers tell the real story: the Decision Tree actually performs worse than the Linear Regression model (an error of ~\$71,000 vs ~\$69,000) due to severe overfitting.

Let's try one last model: the Random Forest Regressor. Random Forests train many Decision Trees on random subsets of features and instances, then average out their predictions. This ensemble technique usually yields much stronger results.

```
[ ]: from sklearn.ensemble import RandomForestRegressor

# Initialize and fit the Random Forest
forest_reg = RandomForestRegressor(n_estimators=100, random_state=42)
forest_reg.fit(housing_prepared, housing_labels)

# Measure training RMSE
forest_predictions = forest_reg.predict(housing_prepared)
forest_mse = mean_squared_error(housing_labels, forest_predictions)
forest_rmse = np.sqrt(forest_mse)
print(f"Random Forest Training RMSE: ${forest_rmse:,.2f}")

# Cross-validate the Random Forest
forest_scores = cross_val_score(forest_reg, housing_prepared, housing_labels,
                                scoring="neg_mean_squared_error", cv=10)
forest_rmse_scores = np.sqrt(-forest_scores)

print("\n--- Random Forest Cross-Validation ---")
display_scores(forest_rmse_scores)
```

Random Forest Training RMSE: \$18,355.53

--- Random Forest Cross-Validation ---

Scores: ['\$94,806.47', '\$47,575.27', '\$65,054.42', '\$56,932.45', '\$60,927.82', '\$60,151.06', '\$46,724.45', '\$79,387.45', '\$74,490.59', '\$49,321.63']

Mean Score: \$63,537.16

Standard Deviation: \$14,672.11

6.4 5.4 Conclusion

Model	Training Error	Validation Error	What is happening?
Linear Regression	\$68k (34.0%)	\$69k (34.5%)	Underfitting: The model is too simple to understand the data.
Decision Tree	\$0 (0.0%)	\$71k (35.0%)	Overfitting: The model memorized the data and fails on new samples.
Random Forest	\$18k (9.0%)	\$k (25.0%)	Best Performance: Still overfitting, but by far the most accurate.

7 6. Fine-Tune Your Model

7.1 6.1 Fine-Tune Your Model via Grid Search

We will use `GridSearchCV` to evaluate different hyperparameter combinations for `RandomForestRegressor`. - The first dictionary evaluates $3 \times 4 = 12$ combinations of `n_estimators` (number of trees) and `max_features` (number of features considered at each split). - The second dictionary evaluates $2 \times 3 = 6$ combinations with `bootstrap` set to `False`. With 5-fold cross-validation (`cv=5`), the model will train a total of $(12 + 6) \times 5 = 90$ times to locate the absolute best configuration.

```
[ ]: from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestRegressor

# Define the parameter grid
param_grid = [
    {'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8]},
    {'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3, 4]},
]

forest_reg = RandomForestRegressor(random_state=42)

# Set up the grid search with 5 folds
grid_search = GridSearchCV(forest_reg, param_grid, cv=5,
                           scoring='neg_mean_squared_error',
                           return_train_score=True)

# Run the grid search (this might take a minute or two)
grid_search.fit(housing_prepared, housing_labels)
```

```
[ ]: GridSearchCV(cv=5, estimator=RandomForestRegressor(random_state=42),
                  param_grid=[{'max_features': [2, 4, 6, 8],
                                'n_estimators': [3, 10, 30]},
                                {'bootstrap': [False], 'max_features': [2, 3, 4],
                                  'n_estimators': [3, 10]}],
                  return_train_score=True, scoring='neg_mean_squared_error')
```

7.2 6.2 Evaluate Best Parameters and Scores

Now we extract the absolute best hyperparameter combination found by the search and print the cross-validation scores for every tested combination to see the progression.

```
[ ]: print("Best Parameters:", grid_search.best_params_)

# Print the best direct estimator object
print("Best Estimator:", grid_search.best_estimator_)

# List all evaluation combinations and their corresponding dollar errors
cvres = grid_search.cv_results_
for mean_score, params in zip(cvres["mean_test_score"], cvres["params"]):
    print(f"${np.sqrt(-mean_score):,.2f}", params)
```

```
Best Parameters: {'max_features': 4, 'n_estimators': 30}
Best Estimator: RandomForestRegressor(max_features=4, n_estimators=30,
random_state=42)
$81,627.87 {'max_features': 2, 'n_estimators': 3}
$72,046.79 {'max_features': 2, 'n_estimators': 10}
$69,538.63 {'max_features': 2, 'n_estimators': 30}
$79,907.16 {'max_features': 4, 'n_estimators': 3}
$70,720.01 {'max_features': 4, 'n_estimators': 10}
$68,102.68 {'max_features': 4, 'n_estimators': 30}
$82,204.84 {'max_features': 6, 'n_estimators': 3}
$71,541.83 {'max_features': 6, 'n_estimators': 10}
$68,744.67 {'max_features': 6, 'n_estimators': 30}
$79,599.11 {'max_features': 8, 'n_estimators': 3}
$70,298.43 {'max_features': 8, 'n_estimators': 10}
$69,188.21 {'max_features': 8, 'n_estimators': 30}
$82,282.67 {'bootstrap': False, 'max_features': 2, 'n_estimators': 3}
$70,793.41 {'bootstrap': False, 'max_features': 2, 'n_estimators': 10}
$77,713.68 {'bootstrap': False, 'max_features': 3, 'n_estimators': 3}
$70,327.98 {'bootstrap': False, 'max_features': 3, 'n_estimators': 10}
$79,473.31 {'bootstrap': False, 'max_features': 4, 'n_estimators': 3}
$71,554.05 {'bootstrap': False, 'max_features': 4, 'n_estimators': 10}
```

8 7. Present Your Solution (our presentation)

8.1 7.1 Save Model & PipeLines

```
[ ]: import joblib

# 1. Extract the best trained model from your grid search
final_model = grid_search.best_estimator_

# 2. Save the model to a file
joblib.dump(final_model, "./production/model.pkl")

# 3. To load it back later (in another notebook or production)
# loaded_model = joblib.load("model.pkl")

# 4. Save pipeline to a file
joblib.dump(full_pipeline, "./production/full_pipeline.pkl")
```

```
[ ]: ['./production/full_pipeline.pkl']
```

9 8. Launch & Monitor

9.1 8.1 Solution 1: The Event-Driven Architecture (FastAPI + Kafka + Ingestion Consumer)

This is the standard enterprise-grade pattern for high-throughput systems. It completely decouples the user-facing prediction path from the analytics and monitoring storage path.

- **Zero I/O Bottlenecks on Predictions:** The user-facing API doesn't wait for database connections or disk writes. It drops the inference logs into an in-memory network buffer (Kafka producer) and returns the HTTP response instantly, keeping prediction latency minimal under massive concurrent load.
- **Absolute Fault Isolation:** If downstream analytical storage (InfluxDB, PostgreSQL, or an S3 data lake) crashes or undergoes maintenance, the core ML API is completely unaffected. Kafka safely retains the data stream until the consumer workers recover.
- **Infinite Replayability & Streaming Analytics:** Because logs are stored sequentially in a distributed commit log, multiple independent services can consume the same data simultaneously. One worker can stream real-time operational aggregates, while another backfills a historical data warehouse.

“mermaid	FastAPI ML Service	(Polls Metrics)	[Prometheus]	[Grafana
] (Runs Inference Only)		(Immediate Sync Response to Client)		(Asyn-
chronous Hand-off)		Kafka / Kinesis Queue	(Buffers spike loads / Prevents	
data loss)		Log Ingestion Worker	(Decoupled consumer writ-	
ten in Go/Python)			S3 / MinIO	Time-Series
DB (Raw Parquet)	(VictoriaMetrics/Influx)			
Airflow/Prefect	(Runs Evidently/Great Expectations)	[Alerting]		

9.2 8.2 Solution 2: The Direct Time-Series Pattern (FastAPI + Direct InfluxDB Writes)

This is a middle-tier, highly efficient architecture for systems that need fast operational performance without the operational complexity of managing a distributed message broker like Kafka.

- **Extremely Low Infrastructure Overhead:** You completely skip the complexity of managing and scaling a distributed message broker cluster (like Kafka/ZooKeeper/Raft). The entire monitoring backend is just a single, highly specialized database.
- **Out-of-the-Box Time Aggregations:** InfluxDB is built explicitly for time-series workloads. Calculating temporal windows—such as calculating a moving average of the model’s predicted house prices or error rates over the last 5 minutes—requires trivial, high-performance queries compared to a standard relational database.
- **Lightweight Non-Blocking I/O:** Using `asyncio` combined with InfluxDB’s native HTTP line protocol allows the API to fire off metrics asynchronously without spinning up heavy execution threads or blocking the execution loop of the web worker.

“mermaid

[Client Request]



9.3 8.3 Solution 3: Micro-Batching File-Drop Pattern (Sidecar Agent)

“mermaid

[Client Request]

